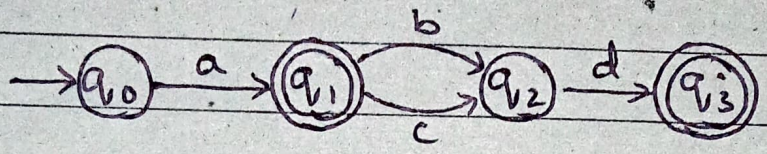
Automata

An automaton is a mechanical device which performs some functions determined by a pre-defined set of instructions. Automata Theory is a branch of Computer Science that deals with designing abstract computing devices that follows a fixed sequence of operations automatically.



An automaton can be represented by a 5-tuple, where

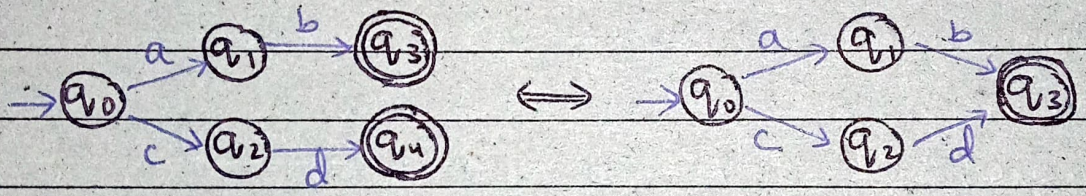
- $Q = \{q_0, q_1, q_2, q_3\}$ // finite set of states
- $\Sigma = \{a, b, c, d\}$ // finite set of alphabets
- $\delta = \{ \delta(q_0, a) = q_1, \delta(q_1, b) = q_2, \delta(q_1, c) = q_2, \delta(q_2, d) = q_3 \}$ // finite set of transition edges
- $q_i = q_0$ // initial state (not a set)
- $F = \{q_1, q_3\}$ // finite set of final states

- String : a finite sequence of symbols taken from Σ.
  length of string is denoted by $|S|$, when $|S|$ we write it as Λ or ε.
  A string S is accepted by a finite automaton $(Q, \Sigma, \delta, q_0, F)$ if $\delta(q_0, S) \Rightarrow F$. i.e., for every input symbol of string it takes one transition and starts with initial state and reaches one of the final states by reading the complete string

- Language : the collection of all accepted strings by an automaton is called the language of the automaton.

- transition function: It takes two inputs — current state and an input signal, and returns and returns the next state.
  eg: $\delta(q_0, a) = q_1$
- extended transition function: It takes two inputs — current state and an string, and returns the state it reaches after reading the string.
  eg: $\delta(q_0, abd) = q_3$

- minimization of a deterministic finite automaton

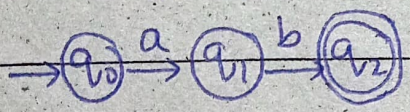


$L_1 = \{ab, cd\}$ = $L_2 = \{ab, cd\}$

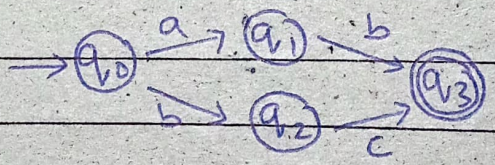
two automata are considered equivalent if they have the exactly same language set.

- some examples of finite automata

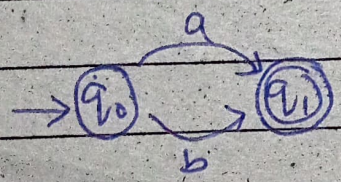
$L = \{ab\}$



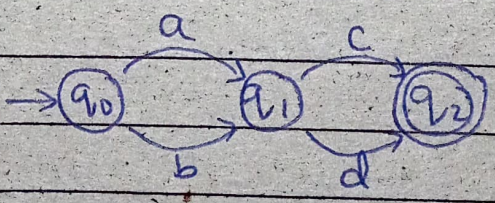
$L = \{ab + cd\}$



$L = \{a + b\}$

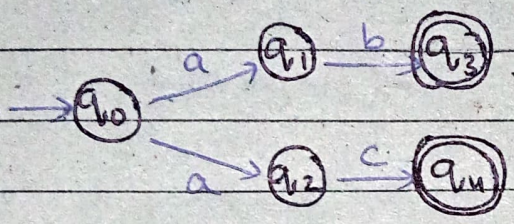


$L = \{(a+b)(c+d)\}$

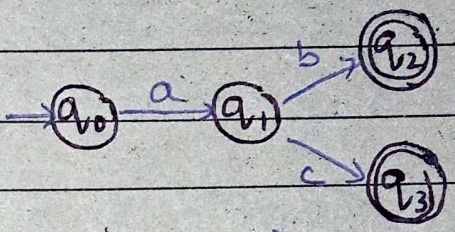


- **Non-deterministic Finite Automata (NFA / NDFA)**

A finite automaton is called NFA, if there exist many paths for a specific input from any state.



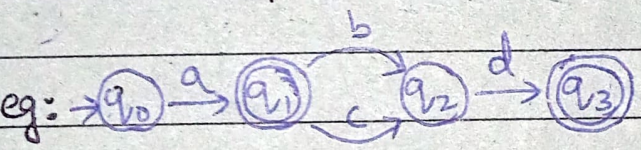
(NFA)



(eq. DFA)

$L = \{a(b+c)\}$

- **Tabular representation of finite automata**

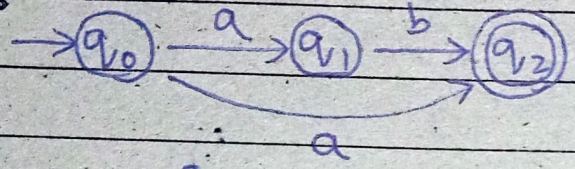


	δ	a	b	c	d
→	q_0	q_1	–	–	–
◎	q_1	–	q_2	q_2	–
	q_2	–	–	–	q_3
◎	q_3	–	–	–	–

q_i	$\delta(q_i, a)$

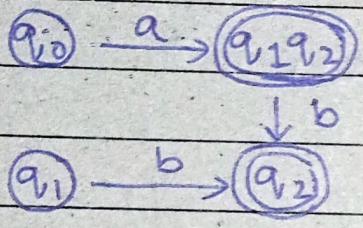
- Conversion from NFA to DFA using table

eg:



$L_1 = \{a, ab\}$

	a	b
→ q_0	q_1, q_2	–
q_1	–	q_2
◎ q_2	–	–
◎ q_1q_2	–	q_2



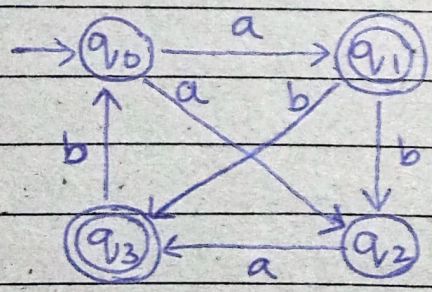
$L_2 = \{a, ab\}$

q_1q_2 inherit properties of both q_1 & q_2

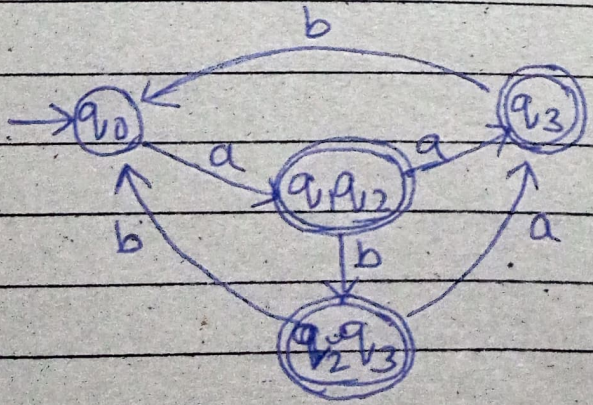
$\because L_1 = L_2$, both are equiv.

In this case, state q_1 can't be reached from the initial state q_0. Hence it has no significance in this automaton.

eg:

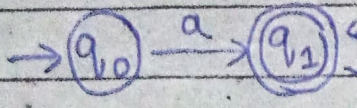


	a	b
→ q_0	q_1q_2	–
◎ q_1	–	q_2q_3
q_2	q_3	–
◎ q_3	–	q_0
◎ q_1q_2	q_3	q_2q_3
◎ q_2q_3	q_3	q_0



States q_1 & q_2 become irrelevant (unreachable).

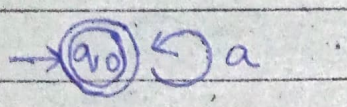
- **Self loops and closures**



$L = \{a, aa, aaa, ...\} = a^+$

$L = a^n \mid n > 0$

(positive closure of a)

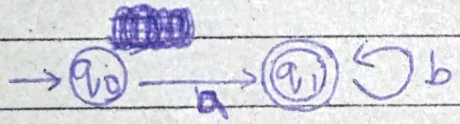


$L = \{\epsilon, a, aa, aaa, ...\} = a^*$

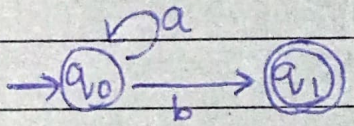
$L = a^n \mid n \geq 0$

(Kleene's closure of a)

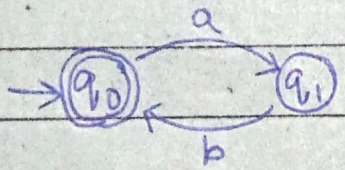
eg: $ab^* = \{a, ab, abb, ...\}$



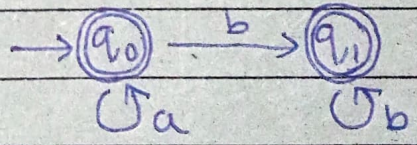
$a^*b = \{b, ab, aab, ...\}$



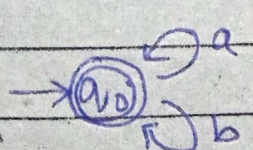
eg: $(ab)^* = \{\epsilon, ab, abab, ...\}$



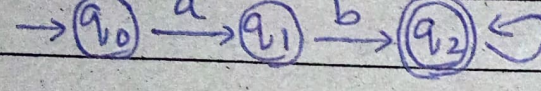
$a^*b^* = \{\epsilon, a, b, ab, aa, bb, ...\}$



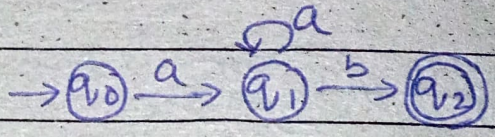
eg: $(a+b)^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, ...\}$



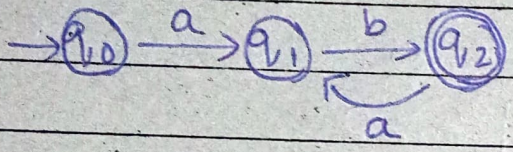
eg: $ab^+ = \{ab, abb, abbb, ...\}$



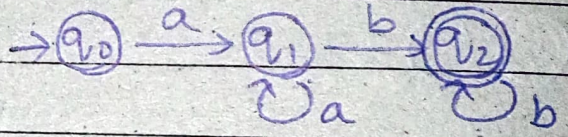
$a^+b = \{ab, aab, aaab, ...\}$



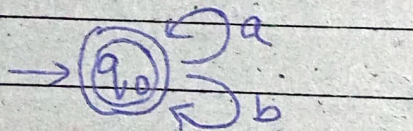
eg: $(ab)^+ = \{ab, abab, ababab, ...\}$



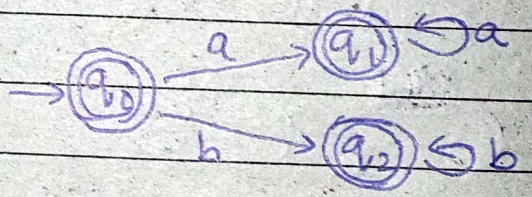
$a^+b^+ = \{ab, aab, abb, aabb, ...\}$



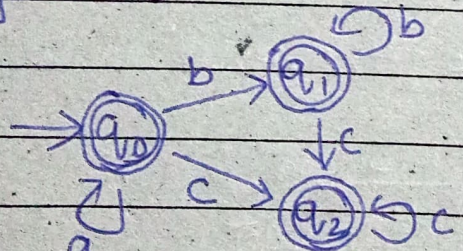
eg: $(a+b)^* = \{\epsilon, ab, b, aa, ab, ba, bb, ...\}$



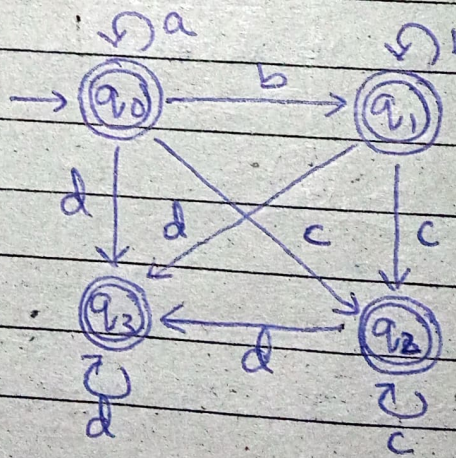
$a^* + b^* = \{\epsilon, a, aa, aaa, b, bb, bbb, ...\}$



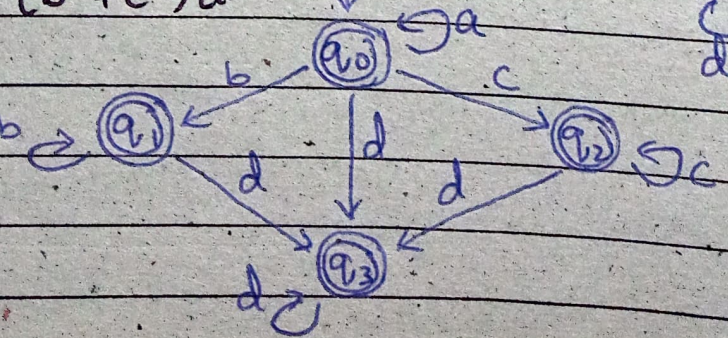
eg: $a^*b^*c^*$



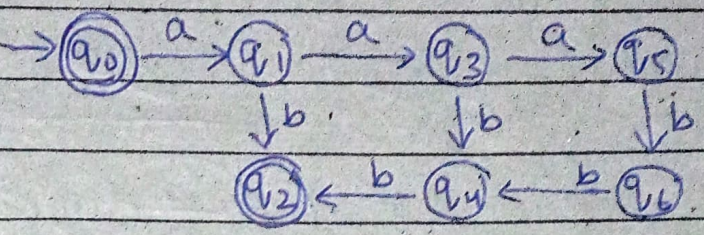
$a^*b^*c^*d^*$



eg: $a^*(b^*+c^*)d^*$



eg: $a^n b^n \mid 0 \leq n < 4$



eg: $a^n b^n \mid 0 \leq n$

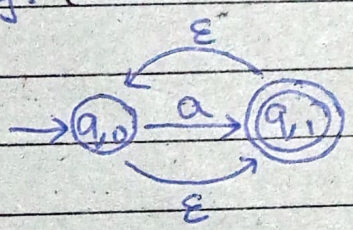
DFA for this language is not possible

a^*b^* is possible but $a^n b^n$ is not. hence if superset is possible it doesn't imply that subset is also possible.

Epsilon NFA (Λ-NFA)

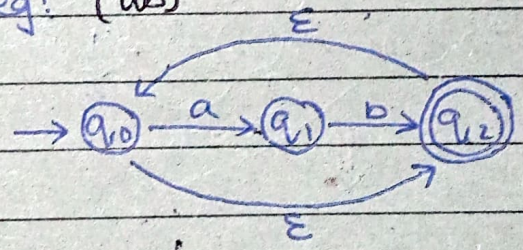
Epsilon NFA is the NFA which contains epsilon moves / Null moves.

eg: ~~aaaa~~ a^*

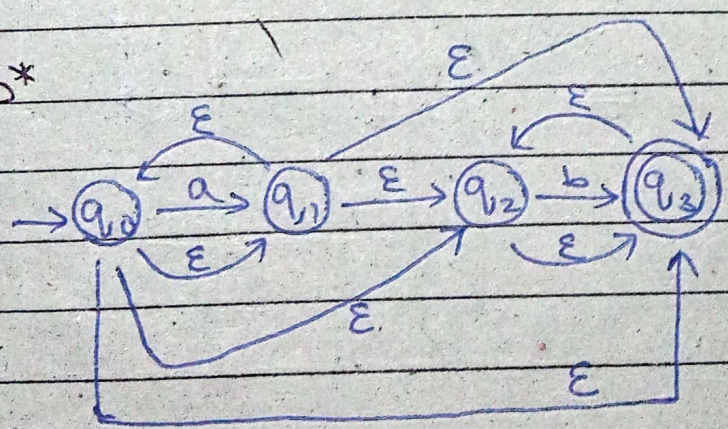


(no self loops)

eg: $(ab)^*$

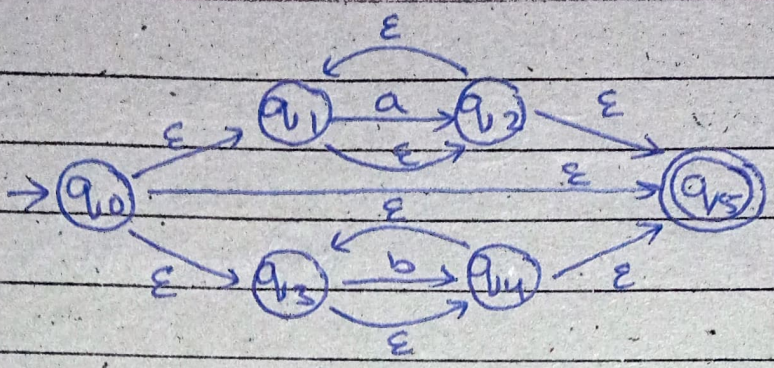


eg: a^*b^*



We must have a direct ε edge from each component to the final state (to show the possible skips).

eg: $a^* + b^*$

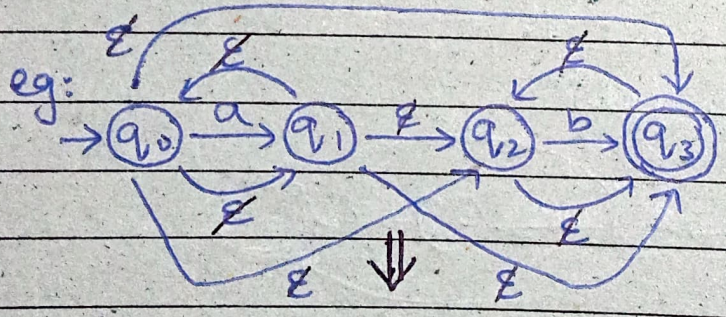


Conversion of Λ-NFA to a transition diagram

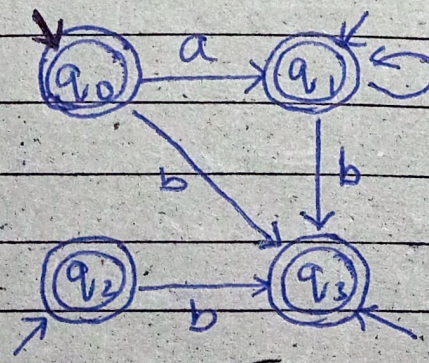
If there exists $q_i \xrightarrow{\epsilon} q_j$, then

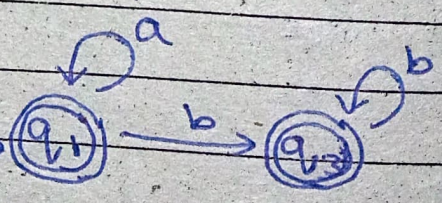
- Duplicate all non-ε edges starting from q_j to now start from q_i.
- If q_i is the initial state, make q_j also an initial state.
- If q_j is a final state, make q_i also a final state.

eg:



Λ-NFA



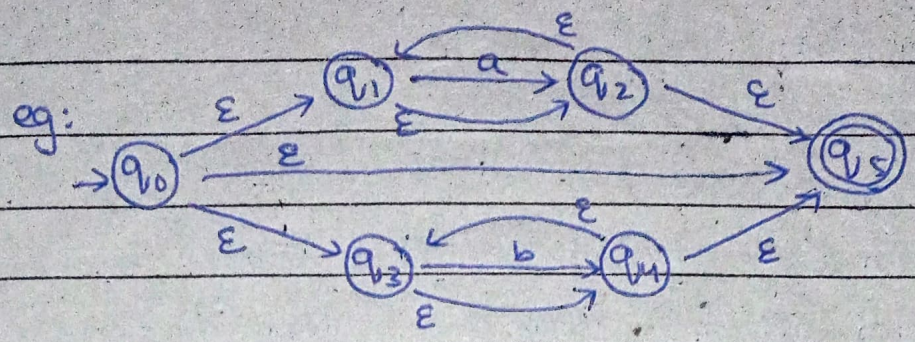


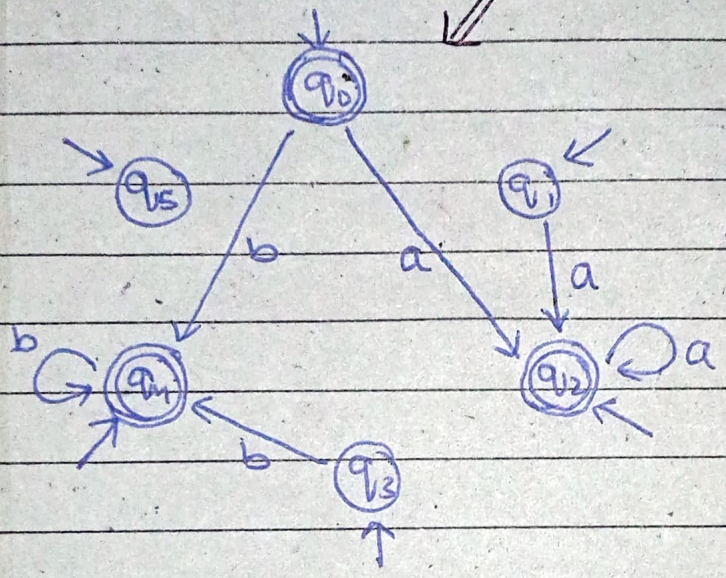
Finite DFA

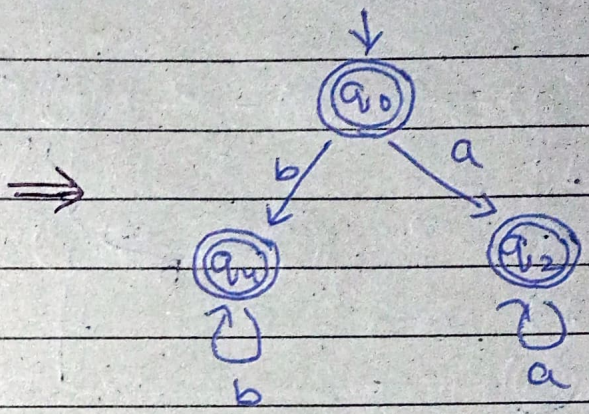
Transition Diagram

Page No.
Date

eg:





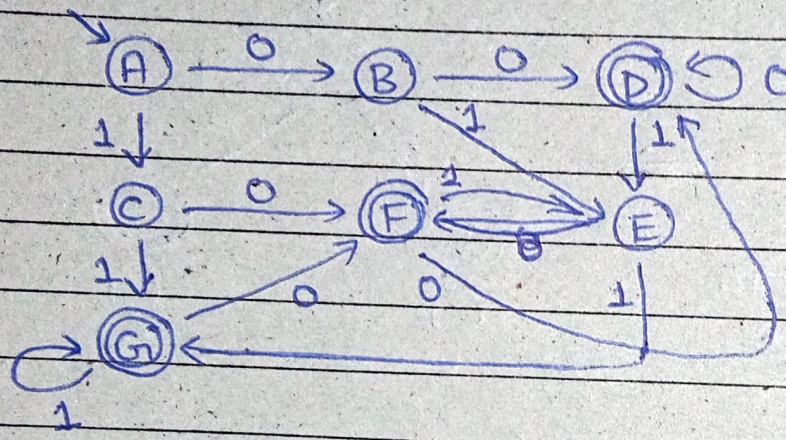


Minimisation of DFA

- Lower triangle matrix method

→ two states having same transitions are considered equal.

→ a final state and a non-final state aren't supposed to be compared.



From Comparison matrix we get

C = E & D = F

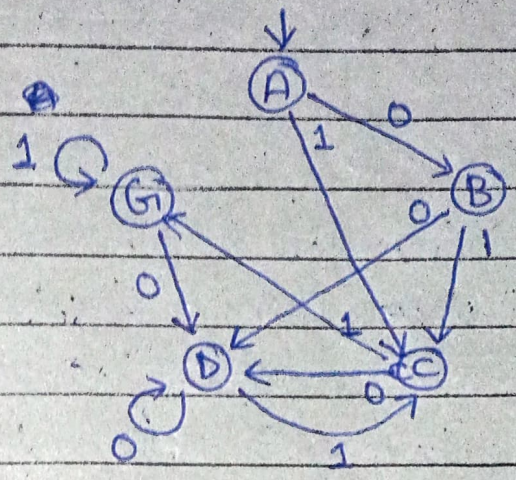
	0	1
A	B	C
B	D	E
C	F	G
D	D	E
E	F	G
F	D	E
G	F	G

	A	B	C	D	E	F	G
A	X	X	X	X	X	X	X
B	X	X	X	X	X	X	X
C	X	X	X	X	X	X	X
D	X	X	X	X	X	X	X
E	X	X	✓	X	X	X	X
F	X	X	X	✓	X	X	X
G	X	X	X	X	X	X	X

Page No.
Date

Hence we will update the transition table as follows:

	0	1
A	B	C
B	D	C
C	D	G
D	D	C
G	D	G



Hence the minimised DFA will look like this ↑.

- equivalence class method.

For the same finite automaton, we will apply equiv. class method.

→ We start with two sets of states, depending on whether they are final or not. i.e., Set α = set of final states, Set β = set of non final.

	α			β			
	{D	F	G}	{A	B	C	E}
0	α	α	α	β	α	β	β
1	β	β	α	β	β	β	β

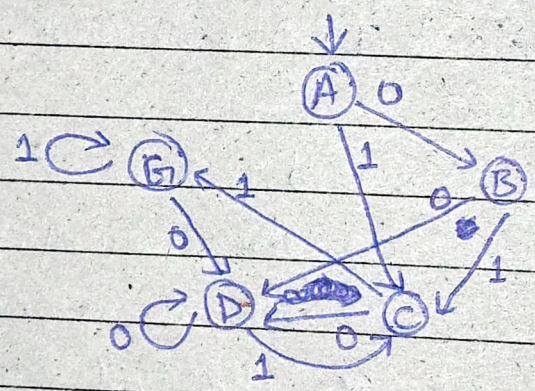
→ now we will keep dividing the sets based on their transitions

	α		β	γ		∂
	{D	F}	{G}	{A	C	E}	{B}
0	α	α	α	∂	α	α	α
1	γ	γ	β	γ	β	β	γ

	α	β	γ	∂	η		
	{D F}	{G}	{A}	{C E}	{B}		
0	α	α	α	η	α	α	α
1	∂	∂	β	∂	β	β	∂

→ at this step no more divisions are possible. hence the states within a same set are equivalent. i.e., here, D≡F & C≡E, so we will replace these, and get the same result as in the previous example.

A	B	C
B	D	C
C	D	G
D	D	C
G	D	G



Finite State Machines (FSM) with Output

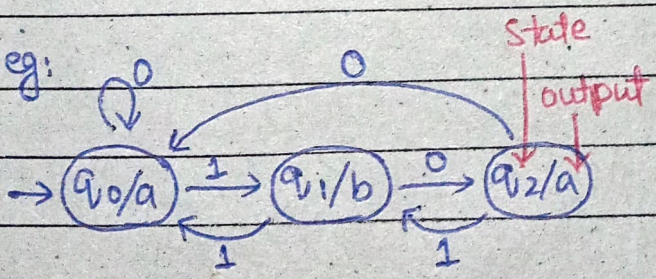
- moore machine
- mealy machine

An FSM with output can be represented by a 6-tuple, where

- Q → finite set of states
- Σ → set of input symbols
- δ → Transition function $(Q \times \Sigma) \rightarrow Q$
- q_0 → initial state
- Δ → set of output symbols
- λ → output function $Q \rightarrow \Delta$ [for moore]
  $(Q \times \Sigma) \rightarrow \Delta$ [for mealy]

Moore machine

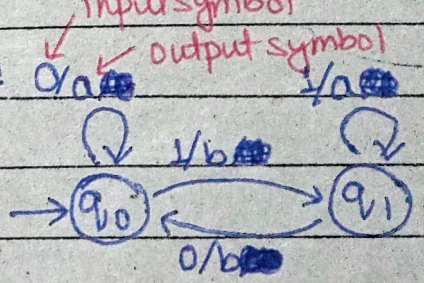
- Output corresponds with the current state
- Output len = Input len + 1
- eg:



input = 10011
output = abaaba

Mealy machine

- Output corresponds with the current transition (i.e. state & input).
- Output len = Input len
- eg:

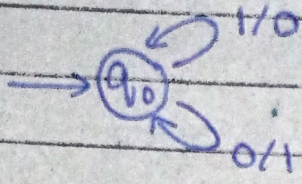


input = 10011
output = bbaba

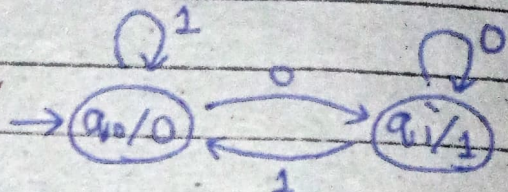
Page No.
Date

Q Create a mealy & moore machine which generates 1's compliment.

soln



mealy



moore